

AI Customer Support Agent

Technical Project Documentation

Project Type	AI-powered customer support application
Primary Technologies	Python, FastAPI, Scikit-learn, Gemini API, HTML, CSS, JavaScript
Knowledge Base	15,000+ customer-support conversations
Intent Categories	9 support intent categories
Author	Sai Ganesh

This document provides a concise technical overview of the project's architecture, machine-learning pipeline, retrieval system, API layer, LLM integration, security approach, setup instructions, and future improvements.

1. Project Overview

The AI Customer Support Agent is a customer-support application that processes a customer's message, identifies the underlying support intent, retrieves relevant historical support guidance, and generates a concise customer-facing response.

The system combines traditional machine-learning techniques with information retrieval and a large language model. A hybrid intent classifier handles common support queries, while TF-IDF and cosine similarity identify relevant support conversations. Google Gemini is then used to turn retrieved guidance into a natural response.

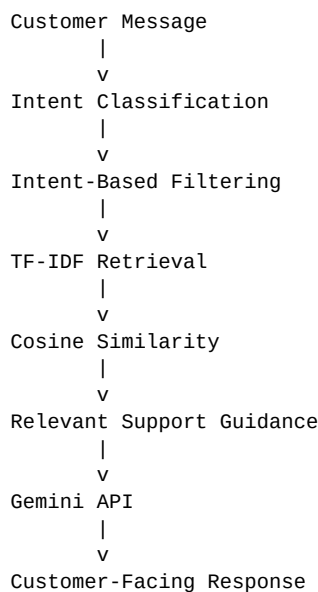
2. Objectives

- Automatically identify the intent behind customer-support messages.
- Retrieve relevant support guidance from a large conversation dataset.
- Generate clear and concise customer-facing responses.
- Provide a REST API that can process support requests in real time.
- Remain usable when the external LLM service is unavailable or rate-limited.

3. Key Features

- Hybrid intent classification using keyword rules and Logistic Regression.
- Nine customer-support intent categories.
- TF-IDF vectorization with cosine-similarity retrieval.
- Knowledge base containing 15,000+ customer-support conversations.
- Google Gemini API integration for natural-language response generation.
- FastAPI REST backend.
- Responsive web-based chat interface.
- Fallback response mechanism for API failures and rate limits.

4. System Architecture



The pipeline first determines the customer's intent. Retrieval is then restricted to conversations belonging to that intent, reducing irrelevant matches. The highest-scoring support response is supplied as factual guidance to the response generator.

5. Technology Stack

Technology	Purpose
Python	Core application, ML and data-processing logic
FastAPI	REST API and backend service
Scikit-learn	TF-IDF vectorization and Logistic Regression
Google Gemini API	Natural-language response generation
HTML / CSS / JavaScript	Web-based chat interface
Git / GitHub	Version control and project hosting

6. Intent Classification

The application supports nine intent categories:

- `internet_wifi_issue` — Internet, Wi-Fi, connectivity and speed problems.
- `tv_fios_content` — TV channels, DVR, streaming, movies and shows.
- `complaint_escalation` — Complaints, manager requests and escalation.
- `equipment_router` — Router, ONT, modem, equipment and status-light issues.
- `installation_availability` — Installation, technician appointments and availability.
- `billing_charges` — Bills, fees, charges, payments and refunds.
- `mobile_phone_issue` — Mobile, LTE, wireless coverage and calling issues.
- `plan_product_upgrade` — Plans, promotions, products and upgrades.
- `account_login` — Account access, login and password issues.

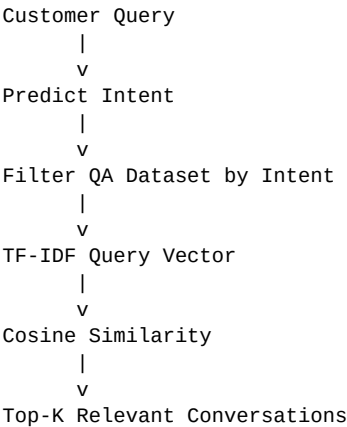
6.1 Hybrid Classification Strategy

A keyword-first layer handles clear, high-signal support terms such as router, Wi-Fi, billing, login and installation. If no keyword rule produces a result, the system falls back to a trained TF-IDF + Logistic Regression model.

```
TfidfVectorizer(  
    ngram_range=(1, 2),  
    sublinear_tf=True,  
    max_features=5000  
)  
  
LogisticRegression(  
    max_iter=2000,  
    C=2.0,  
    class_weight="balanced"  
)
```

7. Information Retrieval

After intent classification, the retrieval layer filters the knowledge base to the predicted intent. TF-IDF vectors are generated for customer messages, and cosine similarity is used to compare the incoming query with candidate conversations.



The highest-scoring retrieved support response is passed to the response generator as factual support guidance.

8. Dataset and Knowledge Base

The project uses a cleaned customer-support QA dataset derived from Verizon support conversations. The final retrieval dataset contains more than 15,000 customer/support message pairs.

Dataset Property	Value
QA pairs	15,986 cleaned pairs
Intent categories	9
Customer messages	15,986
Exact duplicate pairs	0
Empty rows	0

9. Gemini Response Generation

Google Gemini is used after retrieval to produce a natural, customer-facing response. The generation prompt instructs the model to use only the retrieved support guidance as factual information.

Response-generation safeguards include:

- Do not invent policies, prices, links, phone numbers or procedures.
- Do not claim an action was completed unless supported.
- Do not promise refunds, discounts, upgrades, appointments or replacements unless supported.
- Do not mention internal AI, retrieval or classification systems.
- Return only the customer-facing response.

10. Reliability and Fallback Handling

The application includes fallback response logic for situations such as API rate limits, temporary service unavailability and other generation errors. If Gemini cannot generate a response, the system uses the retrieved

support guidance with an intent-specific customer-friendly prefix.

This design prevents a temporary external API problem from completely breaking the support application.

11. FastAPI REST API

The backend exposes a health-check endpoint and a customer-support chat endpoint.

11.1 Health Check

```
GET /health

{
  "status": "online",
  "service": "AI Support API"
}
```

11.2 Customer Support

```
POST /api/chat
```

Example request:

```
{
  "message": "My bill is too high"
}
```

Example response structure:

```
{
  "message": "My bill is too high",
  "intent": "billing_charges",
  "response": "..."
}
```

12. Web Interface

The project includes a responsive web-based chat interface connected directly to the FastAPI backend. The interface provides suggested queries, customer and assistant message bubbles, a typing indicator, and real-time API responses.

13. Project Structure

```
hiver-ai-support-agent/
|
+-- data/
|   +-- processed/
|   +-- ...
|
+-- src/
|   +-- intent/
|   |   +-- classifier.py
|   |   +-- prompts.py
|   |   +-- train_classifier.py
|   |
|   +-- retrieval.py
|   +-- response_generator.py
|   +-- support_agent.py
|
+-- static/
|   +-- index.html
|
+-- api.py
+-- .gitignore
+-- README.md
```

14. Installation and Setup

1. Clone the repository:

```
git clone https://github.com/bhavanasisaiganesh77-hash/ai-customer-support-agent.git
cd ai-customer-support-agent
```

2. Create a virtual environment:

```
python -m venv .venv
```

3. Activate it on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

4. Install project dependencies:

```
pip install -r requirements.txt
```

5. Configure the Gemini API key as an environment variable:

```
$env:GEMINI_API_KEY="YOUR_API_KEY"
```

6. Start the FastAPI server:

```
python -m uvicorn api:app --reload --port 8000
```

7. Open the application at:

```
http://127.0.0.1:8000
```

15. Security

- API keys are stored through environment variables rather than source code.
- Secret and environment files are excluded through .gitignore.
- Actual credentials must never be committed to GitHub.
- The response generator is instructed not to fabricate sensitive support details.

16. Example Queries

Customer Query	Expected Intent
My bill is too high	billing_charges
My internet is not working	internet_wifi_issue
My router has a red light	equipment_router
I cannot log into my account	account_login

17. Future Improvements

- Add multi-turn conversation history and context management.
- Replace or augment TF-IDF with modern vector embeddings.
- Add automated evaluation metrics for retrieval and response quality.
- Expand unit and integration test coverage.
- Add Docker-based deployment.
- Add monitoring, structured logging and observability.
- Support additional customer-support knowledge bases.

18. Repository

GitHub repository: <https://github.com/bhavanasisaiganesh77-hash/ai-customer-support-agent>

19. Author

Sai Ganesh

B.Tech Computer Science Engineering

GitHub: [bhavanasisaiganesh77-hash](#)

Note: This documentation intentionally does not contain any private API keys or credentials.